1 JSP

1.1 Objective

- Introduction to JSP
- Lifecyle of JSP
- Basic Syntax of Scripting Elements
- Implicit objects
- Understand how to interact between multiple jsps
- Understand exception handling in JSP
- Understand JSP usebean tag
- Understand JSP single thread model

1.1.1 Content

1.1.2 Introduction

As we know, we write HTML content in the Servlet to produce HTTP response. To code a servlet is tidious and complex task if solution is complex and large. All the HTML content represents view or presentation of the information processed in Servlet. It would be better if we can separate the view/presentation from the Servlet. This separation results in new technology called as Java Server Pages(JSP) , where JAVA code is written inside HTML. Following is example of servlet and JSP performing the same task.

Snippet of HelloTCSServlet.java

```
/**
 * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse
 *      response)
 */
protected void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {

    PrintWriter out = response.getWriter();
    out
            .println("<!DOCTYPE html PUBLIC \"-//W3C//DTD HTML 4.01 Transitional//EN\"\"http://www.w3.org/TR/html4/loose.dtd\">\n"
                    + "<html> \n"
                    + "<head> \n"
                    + "<meta http-equiv=\"Content-Type\" content=\"text/html;  charset=ISO-8859-1\"> \n"
                    + "<title> My first jsp  </title> \n"
                    + "</head> \n"
                    + "<body> \n");
    out.println("Hello TCS");
    out.println("</body> \n" + "</html>");
}
```

Snippet of Welcome.jsp

```
HelloTCSServlet.java    welcome.jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/l
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
Hello TCS
</body>
</html>
```

You might have observed the "welcome.jsp" is nothing but full of HTML elements and text. Because it is static page, thus does not have any dynamic content generation elements inside it. However first two lines of the code is something different which we do not write in an HTML. We will talk about those lines later in the chapter.

As there is a need to have content which is generated dynamically based on user's request, we need to have some mechanism to have a capability to produce dynamic content in JSP. E.g. Lets say in the above example we want "Hello TCS" to be changed to "Hello <user>" where <user> is a name of user who is accessing the web page so that when a user is viewing the page he would be seeing something like "Hello Suyog". Another example is when you logged in to your "Gmail" account, you see your detail on the home page such as your login name, your emails, your contact lists etc.. It happens for every person who logs into Gmail, the layout is same for all, application is same for all but content is dynamic based on the user who logs into the application.

As we know that Server can execute any servlet depending on the request received. Since JSP also provides us the capability to generate the content dynamically, it is required to be executed on the Server. That means there should be some relation between JSP and Servlet. When we create a JSP and executes it on a Server, the Server creates a Servlet corresponding to the JSP. Following is the servlet(a java class) generated by the Server for "welcome.jsp".

```
welcome_jsp.java   new 2
1   package org.apache.jsp;
2
3   import javax.servlet.*;
4   import javax.servlet.http.*;
5   import javax.servlet.jsp.*;
6
7   public final class welcome_jsp extends org.apache.jasper.runtime.HttpJspBase
8       implements org.apache.jasper.runtime.JspSourceDependent {
9
10    private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
11
12    private static java.util.List _jspx_dependants;
13
14    private javax.el.ExpressionFactory _el_expressionfactory;
15    private org.apache.AnnotationProcessor _jsp_annotationprocessor;
16
17    public Object getDependants() {
18      return _jspx_dependants;
19    }
20
21    public void _jspInit() {
22      _el_expressionfactory = _jspxFactory.getJspApplicationContext(getServletConfig().getServletContext()).getExpressionFactory();
23      _jsp_annotationprocessor = (org.apache.AnnotationProcessor) getServletConfig().getServletContext().getAttribute(org.apache.AnnotationProcessor.class.getName());
24    }
25
26    public void _jspDestroy() {
27    }
28
29    public void _jspService(HttpServletRequest request, HttpServletResponse response)
30        throws java.io.IOException, ServletException {
31
32      PageContext pageContext = null;
33      HttpSession session = null;
34      ServletContext application = null;
35      ServletConfig config = null;
36      JspWriter out = null;
37      Object page = this;
38      JspWriter _jspx_out = null;
39      PageContext _jspx_page_context = null;
```

```
welcome_jsp.java   new 2
38      JspWriter _jspx_out = null;
39      PageContext _jspx_page_context = null;
40
41      try {
42        response.setContentType("text/html; charset=ISO-8859-1");
43        pageContext = _jspxFactory.getPageContext(this, request, response,
44                  null, true, 8192, true);
45        _jspx_page_context = pageContext;
46        application = pageContext.getServletContext();
47        config = pageContext.getServletConfig();
48        session = pageContext.getSession();
49        out = pageContext.getOut();
50        _jspx_out = out;
51
52        out.write("\r\n");
53        out.write("<!DOCTYPE html PUBLIC \"-//W3C//DTD HTML 4.01 Transitional//EN\" \"http://www.w3.org/TR/html4/loose.dtd\">\r\n");
54        out.write("<html>\r\n");
55        out.write("<head>\r\n");
56        out.write("<meta http-equiv=\"Content-Type\" content=\"text/html; charset=ISO-8859-1\">\r\n");
57        out.write("<title>Welcome to TCS</title>\r\n");
58        out.write("</head>\r\n");
59        out.write("<body>\r\n");
60        out.write("\r\n");
61        out.write("Hello TCS\r\n");
62        out.write("\r\n");
63        out.write("</body>\r\n");
64        out.write("</html>");
65      } catch (Throwable t) {
66        if (!(t instanceof SkipPageException)){
67          out = _jspx_out;
68          if (out != null && out.getBufferSize() != 0)
69            try { out.clearBuffer(); } catch (java.io.IOException e) {}
70          if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
71        }
72      } finally {
73        _jspxFactory.releasePageContext(_jspx_page_context);
74      }
75    }
76  }
```

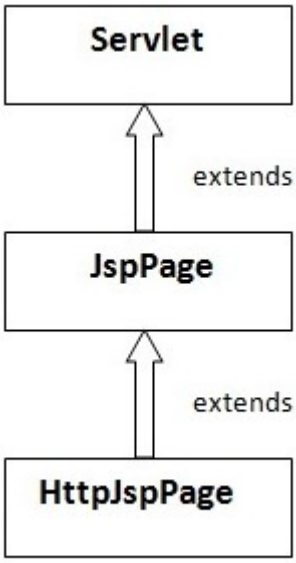
You can see the above generated servlet has similar methods to what we usually have in a servlet (recall servlet lifecyle methods). Also you can see the HTML elements are available in the generated servlet. Detail about all these methods are mentioned later in the chapter.

Note:- welcome_jsp.java is not created by you, its created by the Server for welcome.jsp, which is created by you. Thus if JSP page is modified, we don't need to recompile and redeploy the project/application. The servlet code gets

updated and recompiled automatically by the Server.

We know that each Servlet has to implmement Servlet interface directly or indirectly, therefore the generated servlet should also implement Servlet interface in some way.

According to the JSP specification, all the generated servlet classes must implement the JspPage interface which extends the Servlet interface.



Notice that the generated servlet extends ***HttpJspBase***; This is the super class of all JSP-generated servlets. The class declaration is below:
public abstract class HttpJspBase ***extends*** *javax.servlet.http.HttpServlet* ***implements*** *javax.servlet.jsp.HttpJspPage*

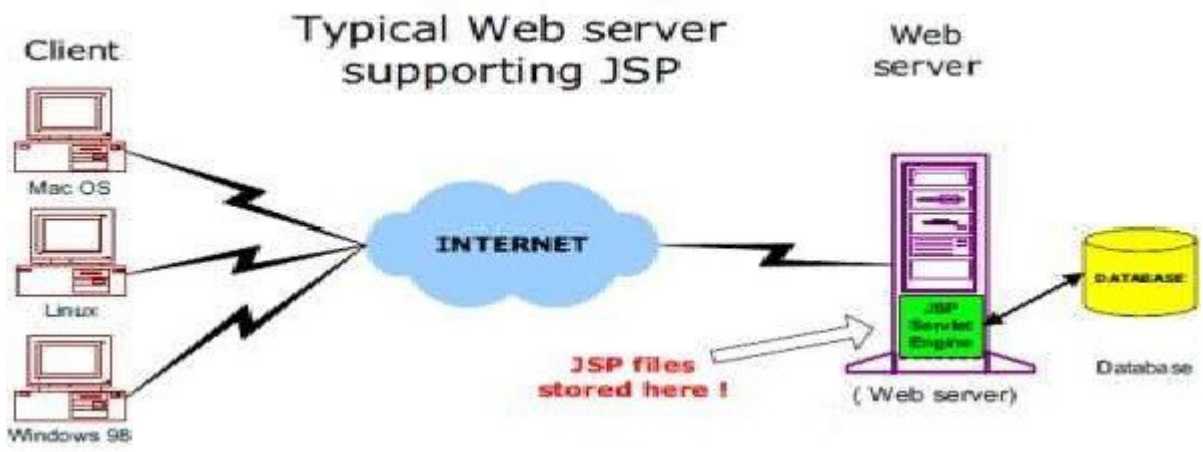
To create dynamic content in JSP, developers insert java code in HTML pages by making use of special JSP tags, most of which start with <% and end with %>.

1.1.3 JSP Architecture:

The web server needs a JSP engine ie. container to process JSP pages(Similar to Servlets). The JSP container is responsible for intercepting

requests for JSP pages.

A JSP container works with the Web server to provide the runtime environment and other services a JSP needs. It knows how to understand the special elements that are part of JSPs.
Following diagram shows the position of JSP container and JSP files in a Web Application.



JSP Processing:
The following steps explain how the web server creates the web page using JSP:

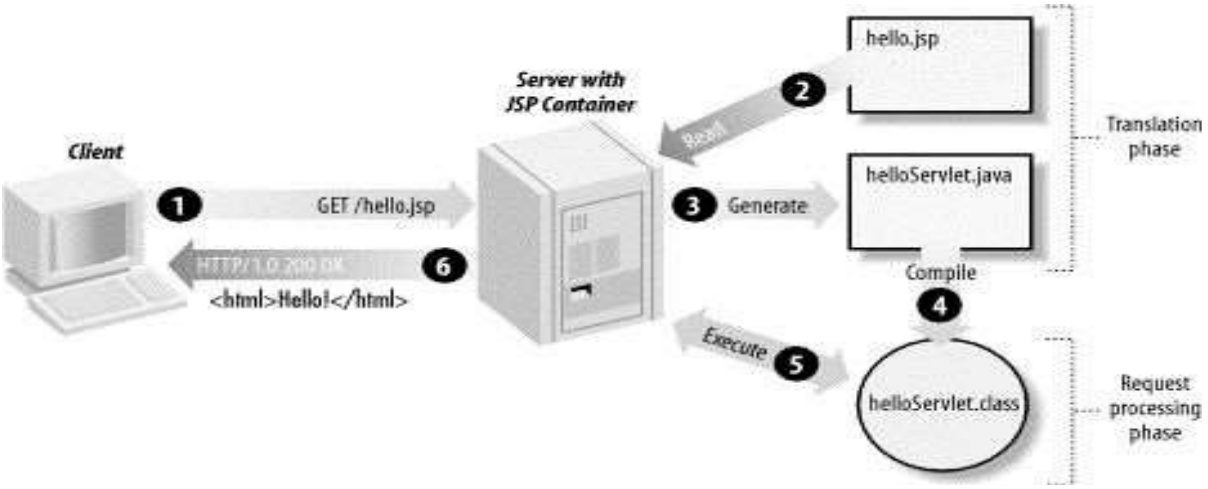
- As with a normal page, your browser sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine.
- The JSP engine loads the JSP page from disk and converts it into a servlet content.

1.1.4 Lifecycle of JSP

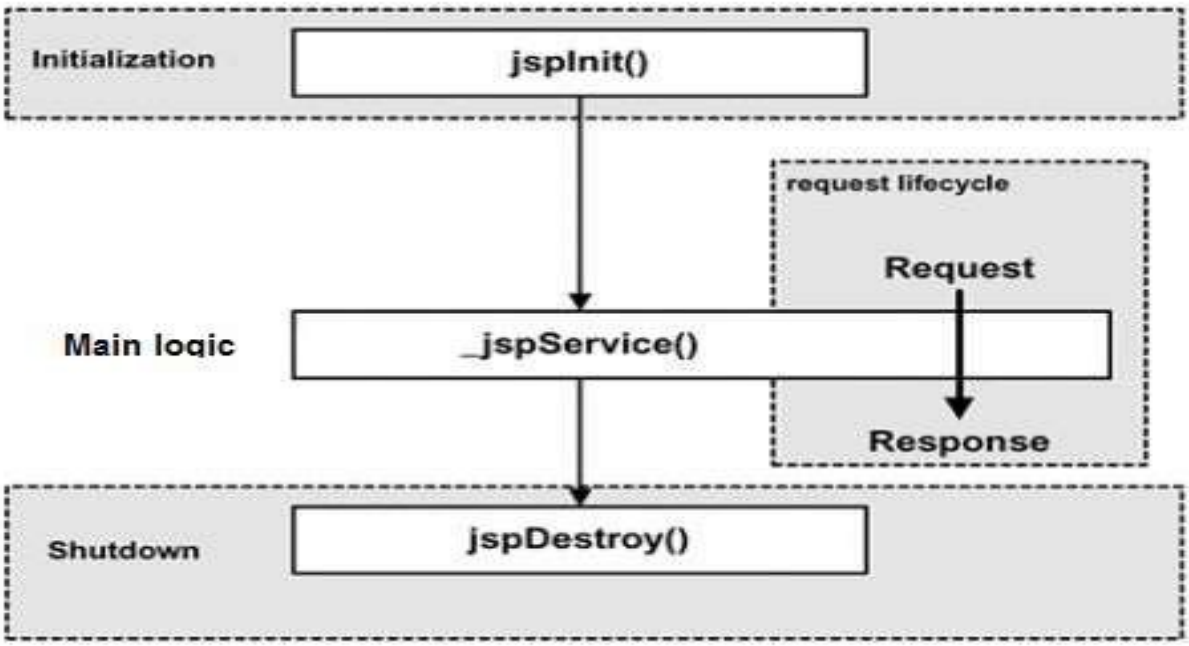
Following are the steps performed by the Server(container) when a user request the JSP. The user will not know about this process as he only needs the HTTP response for the request.

- Translation of JSP Page into corresponding Servlet.
- Compilation of JSP Page meaning compliation Generated Servlet program.

- Classloading (class file is loaded by the classloader)
- Instantiation (Object of the Generated Servlet is created).
- Initialization (jspInit() method is invoked by the container).
- Execution (_jspService() method is invoked by the container).
- Destroy (jspDestroy() method is invoked by the container).If you have noticed, a JSP life cycle can be defined as the entire process from its creation till the destruction which is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet. Following diagram illustrates the same behaviour.



Similar to Servlet, Initialization and Destroy steps are executed once per lifecyle. Following diagram illustrates the methods corresponding to these steps that will be available in the generated Servlet.



JSP Initialization:

When a container loads a JSP it invokes the *_jspInit*() method before servicing any requests. If you need to perform JSP-specific initialization, override the *_jspInit*() method:

```
public void jspInit(){
// Initialization code...
}
```

Typically initialization is performed only once and as with the servlet init method, you generally initialize database connections, open files, and create lookup tables in the jspInit method.

JSP Execution:

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed.
Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the ***_jspService*()** method in the JSP.

The *_jspService*() method takes objects of an ***HttpServletRequest*** and of an ***HttpServletResponse*** as its parameters as follows:

```
void _jspService(HttpServletRequest request,
HttpServletResponse response)
{
// Service handling code...
}
```

The *_jspService*() method of a JSP is invoked once per a request and is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods ie. GET, POST, DELETE etc.

Destroy:

The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container.
The ***_jspDestroy*()** method is the JSP equivalent of the destroy method for servlets. Override _jspDestroy when you need to perform any cleanup, such as releasing database connections or closing open files.
The *_jspDestroy*() method has the following form:

```
public void jspDestroy()
```

```
{
// Your cleanup code goes here.
}
```

1.1.5 Basic Syntax

The scripting elements provides the ability to insert java code inside the jsp. There are three types of scripting elements:

- scriptlet tag
- expression tag
- declaration tag.

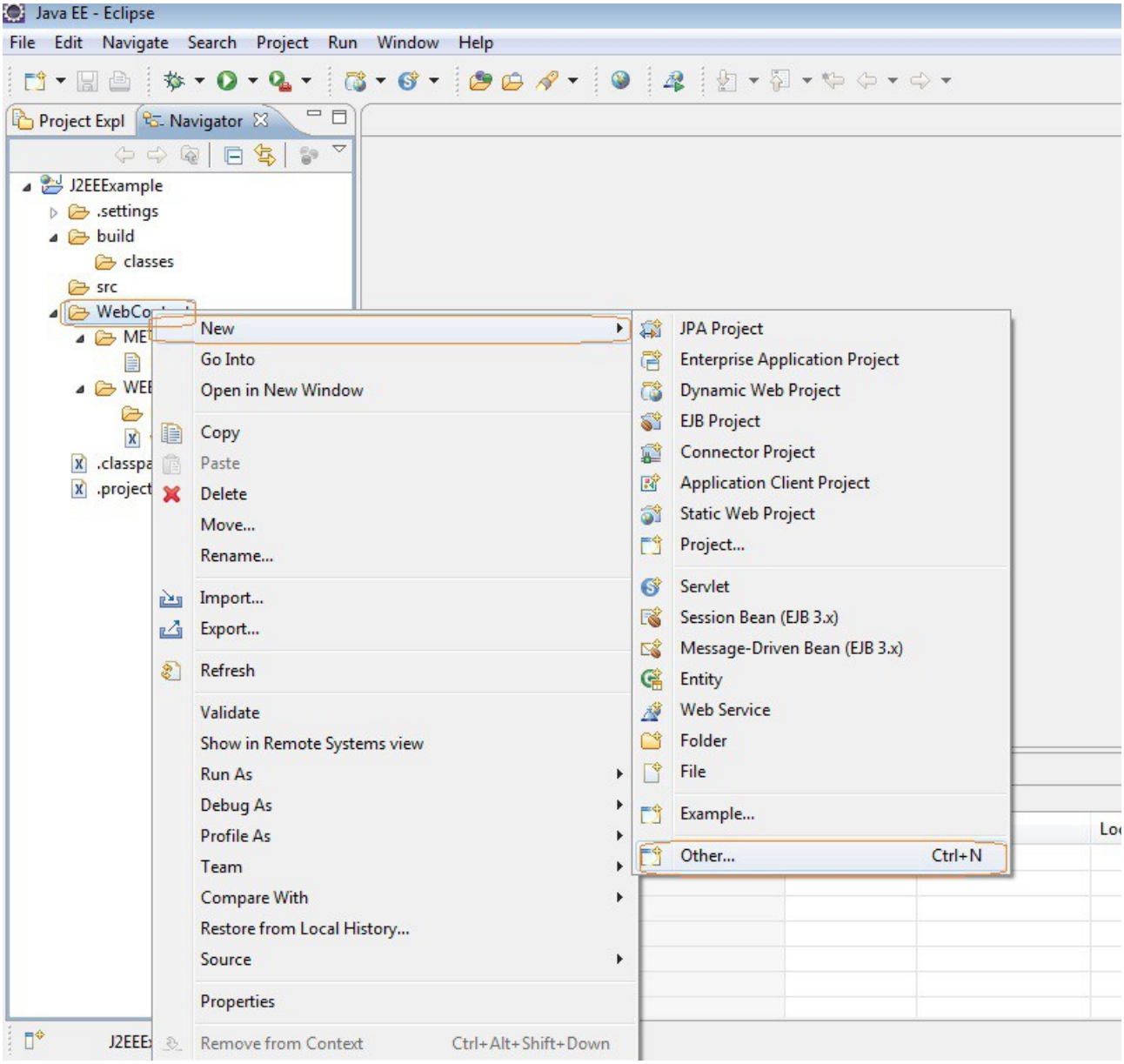
JSP scriptlet tag:

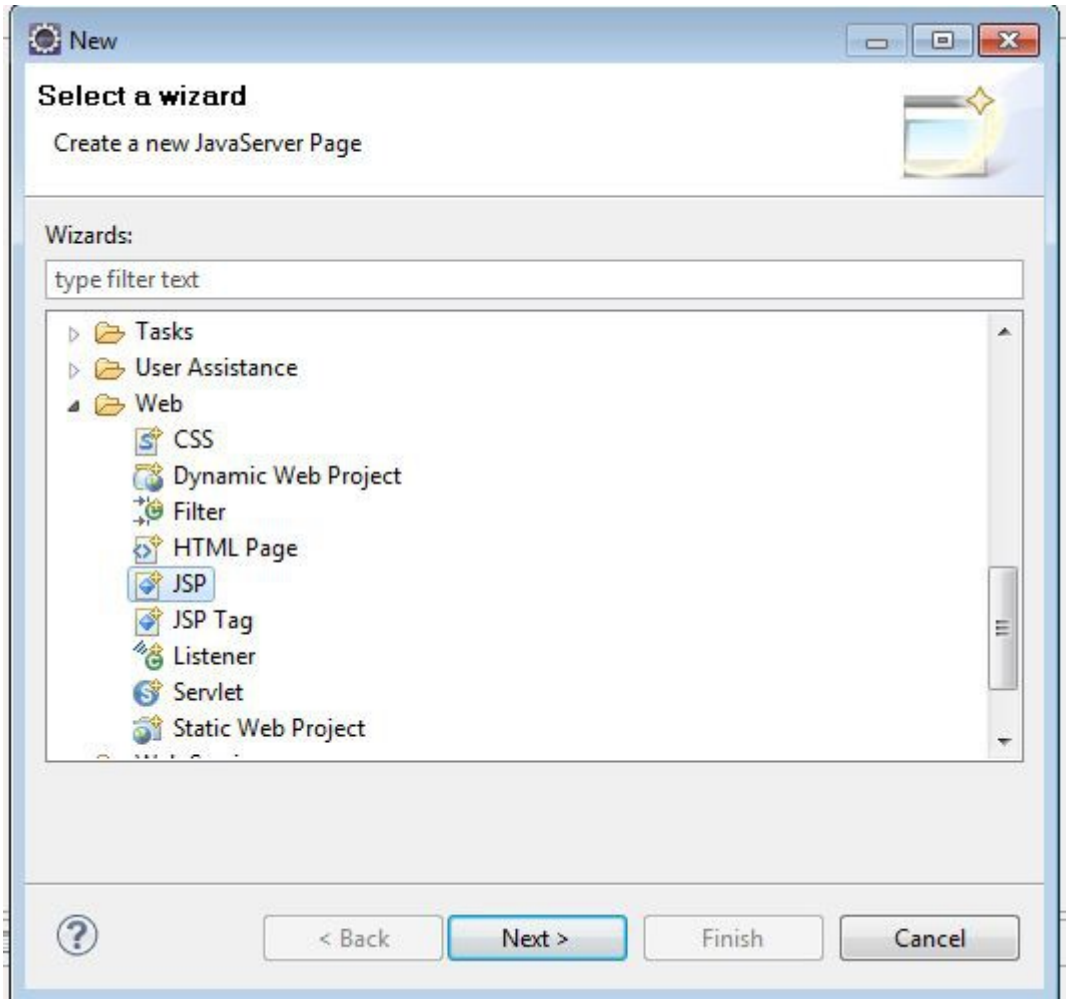
A scriptlet tag is used to execute java source code in JSP. Syntax is as follows:

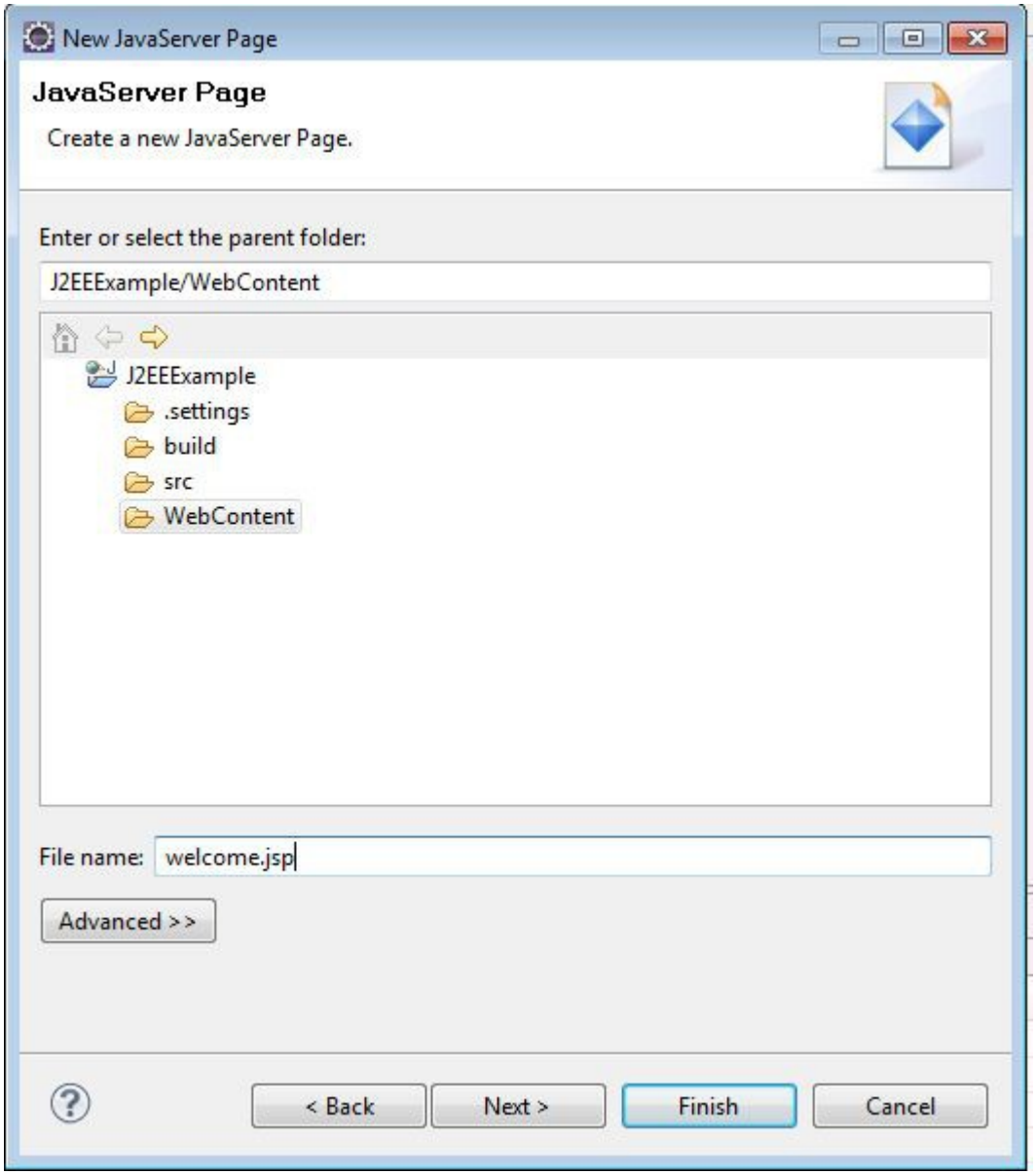
```
<%  java source code %>
```

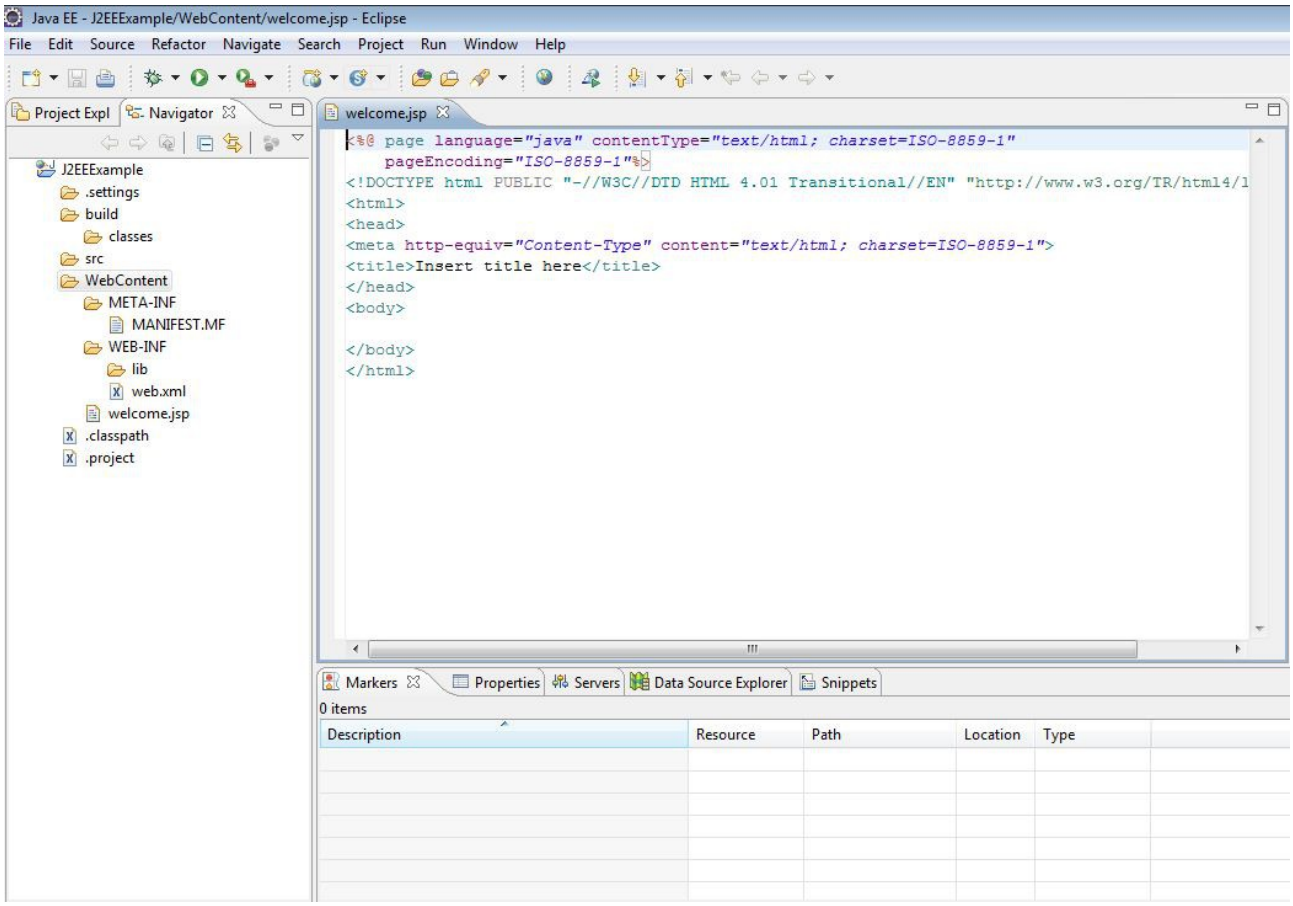
Each Java statement in the scriptlet tag follows rules of Java such as end of a statement with semicolon (;). All the code written inside scriptlet tag will be part of _jsp*service*() method of auto generated servlet.

Example-1: For this example only additional screenshot of Eclipse are there

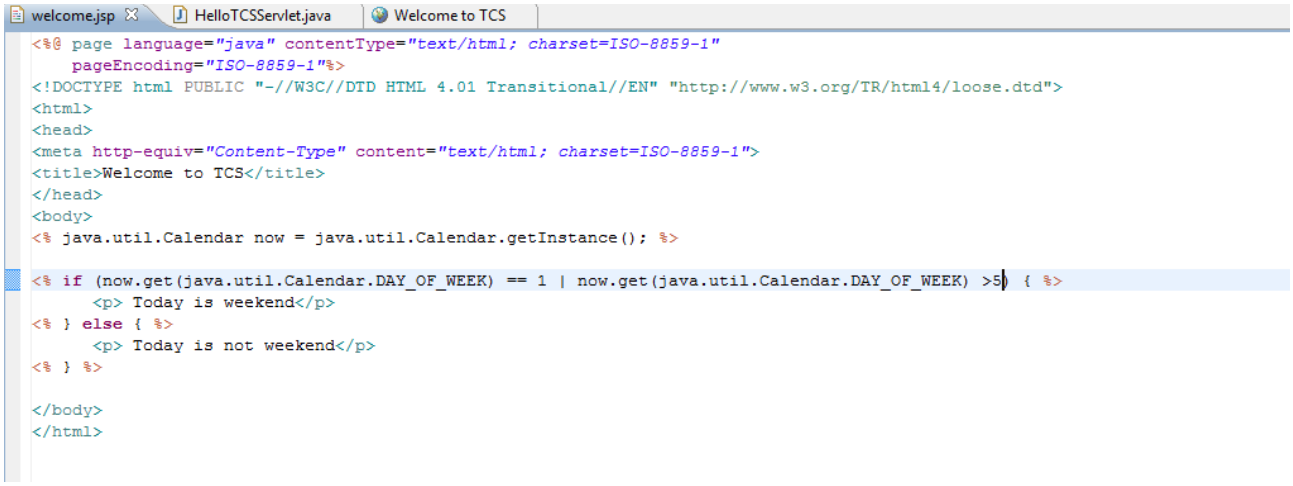

Java EE - Eclipse
File Edit Navigate Search Project Run Window Help
Project Expl
Navigator
J2EEExample
.settings
build
classes
src
New
Go Into
Open in New Window
Copy
Paste
Delete
Move...
Rename...
Import...
Export...
Refresh
Validate
Show in Remote Systems view
Run As
Debug As
Profile As
Team
Compare With
Restore from Local History...
Source
Properties
Remove from Context
Ctrl+Alt+Shift+Down
JPA Project
Enterprise Application Project
Dynamic Web Project
EJB Project
Connector Project
Application Client Project
Static Web Project
Project...
Servlet
Session Bean (EJB 3.x)
Message-Driven Bean (EJB 3.x)
Entity
Web Service
Folder
File
Example...
Other...
Ctrl+N


New
Select a wizard
Create a new JavaServer Page
Wizards:
type filter text
Tasks
User Assistance
Web
CSS
Dynamic Web Project
Filter
HTML Page
JSP
JSP Tag
Listener
Servlet
Static Web Project
< Back
Next >
Finish
Cancel

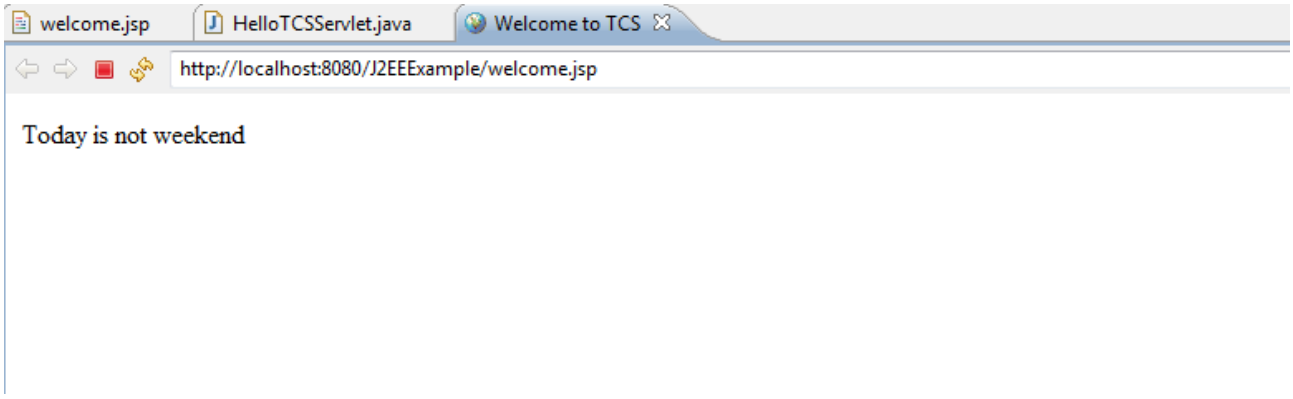

New JavaServer Page
JavaServer Page
Create a new JavaServer Page.
Enter or select the parent folder:
J2EEExample/WebContent
J2EEExample
.settings
build
src
WebContent
File name:
welcome.jsp
Advanced >>
< Back
Next >
Finish
Cancel



welcome.jsp



Output-1:



JSP Expression tag:

The code placed within expression tag is written to the output stream of the response. So you need not write *out.print()* to write data. It is mainly used to print the values of variable or values returned by the method. Syntax is as follows:

```
<%=  java statement %>
```

Here statement does not end with semicolon(;). Also the statement must be evaluated to a String which gets written to the output stream. All the code written in expression tag will be converted into *out*.*println*() statements of *service*() method of auto generated servlet.

Example-1:

```
welcome.jsp   HelloTCSServlet.java   Welcome to TCS
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Welcome to TCS</title>
</head>
<body>

Today's date is <%= new java.util.Date().toString() %>

<br></br>

<% java.util.Calendar now = java.util.Calendar.getInstance(); %>

<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
      <p> Today is weekend</p>
<% } else { %>
      <p> Today is not weekend</p>
<% } %>

</body>
</html>
```

Output-1:

```
welcome.jsp   HelloTCSServlet.java   Welcome to TCS
http://localhost:8080/J2EEExample/welcome.jsp

Today's date is Wed Jun 04 16:21:25 IST 2014

Today is not weekend
```

JSP Declaration tag:

The JSP declaration tag is used to declare fields and methods.

The code written inside the jsp declaration tag is placed outside the service() method of auto generated servlet. Syntax is as follows:

```
<%!  field or method declaration %>
```

All the code which is defined in the Declaration tag is considered at class level

(attributes of the Servlet class). That means, the code written inside the jsp declaration tag is placed outside the service() method of auto generated servlet.

Example-1

```
welcome.jsp    HelloTCSServlet.java    Welcome to TCS

<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Welcome to TCS</title>
</head>
<body>

<%! String orgName = "TCS" ; %>

Hello <%= orgName.toString() %>

Today's date is <%= new java.util.Date().toString() %>

<br></br>

<% java.util.Calendar now = java.util.Calendar.getInstance(); %>

<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
      <p> Today is weekend</p>
<% } else { %>
      <p> Today is not weekend</p>
<% } %>

</body>
</html>
```

Output-1

```
welcome.jsp    HelloTCSServlet.java    Welcome to TCS

http://localhost:8080/J2EEExample/welcome.jsp

Hello TCS

Today's date is Wed Jun 04 16:52:29 IST 2014

Today is not weekend
```

1.1.6 JSP Comments

Adding comments to a JSP page is good practice. There are two types of

comments that can appear in a JSP page:

1. JSP comments, which are comments documenting what the page is doing.
2. HTML/XHTML comments, which are comments that will be sent to the browser.

A JSP comment starts with *<%-- and ends with --%>*. For instance, the following is a JSP comment:

```
<%-- retrieve products to display --%>
```

A JSP comment is not sent to the browser and cannot be nested.
An HTML/XHTML comment has the following syntax:

```
<!-- [comments here] -->
```

An HTML/XHTML comment is not processed by the container and is sent to the browser as is. One use of the HTML/XHTML comment is to identify the JSP page itself:

```
<!-- this is /jsp/store/displayProducts.jspf -->
```

This is particularly useful when working with an application that has many JSP fragments. The developer can easily find out which JSP page or fragment generated a certain HTML section by viewing the HTML source in the browser.

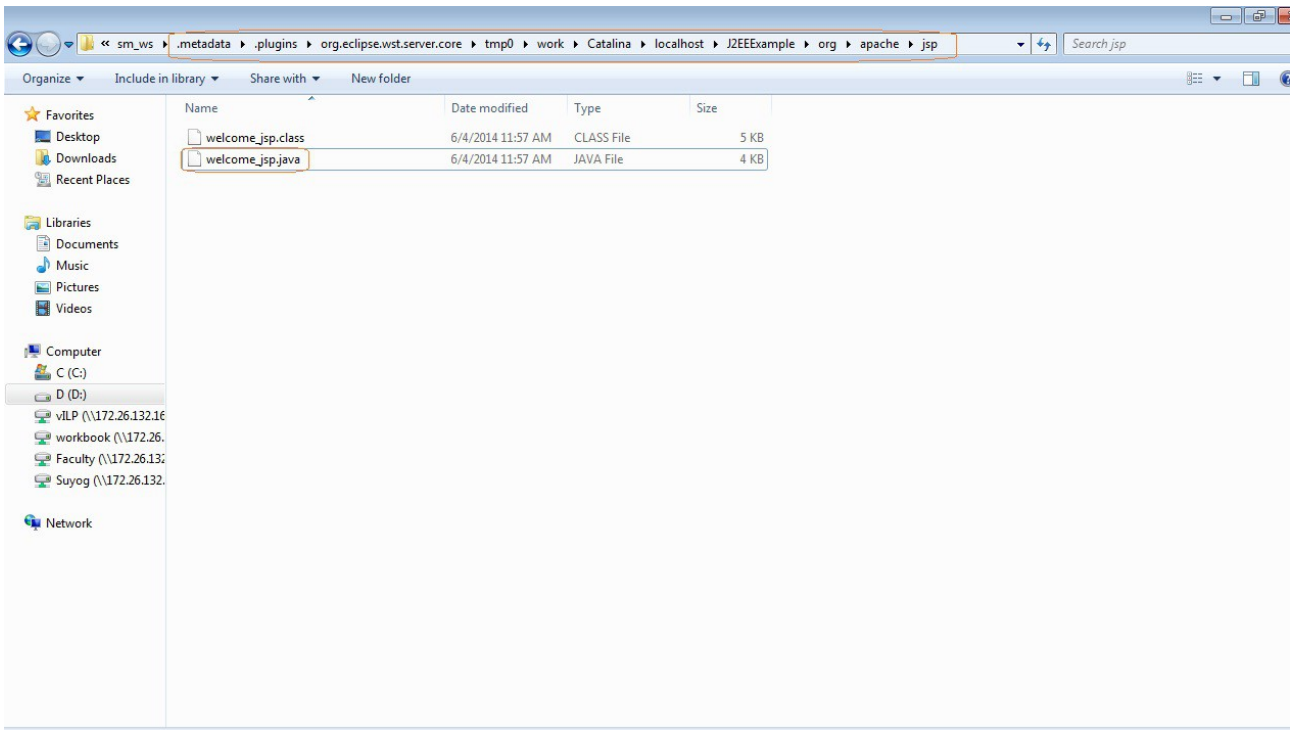
Example-1

welcome.jsp | HelloTCSServlet.java | Welcome to TCS

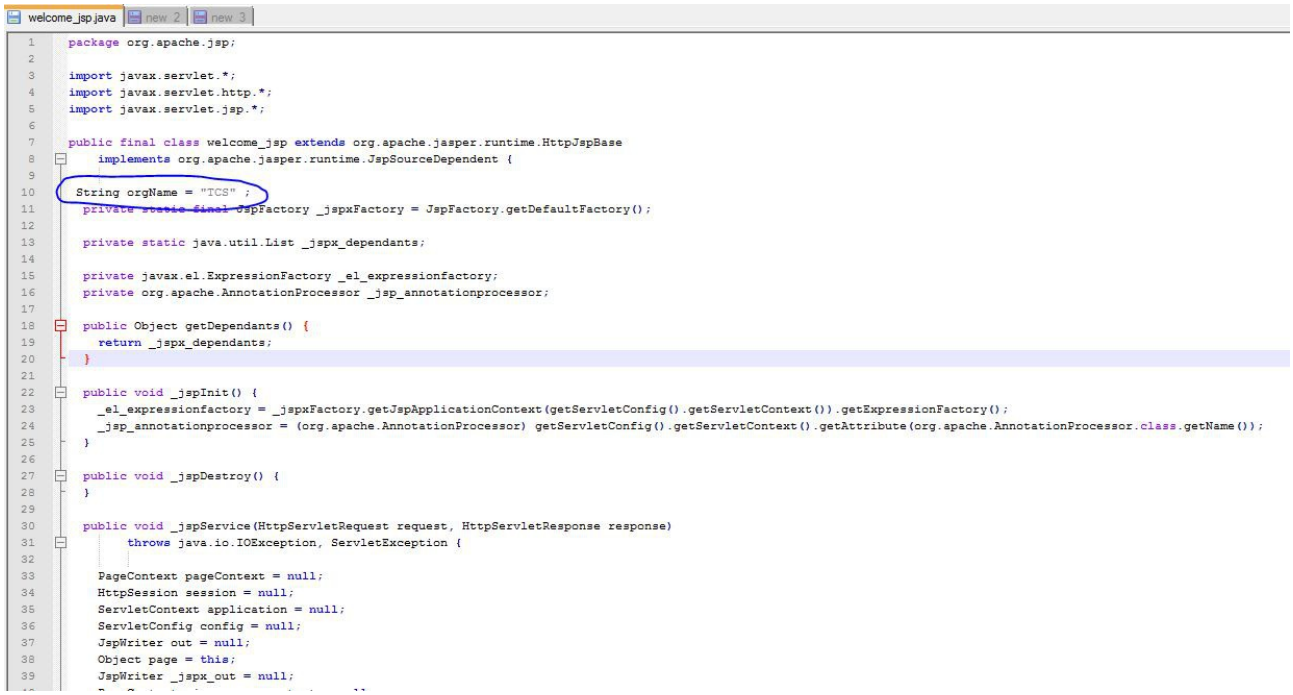
```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Welcome to TCS</title>
</head>
<body>
<!-- JSP example created by Suyog Mittal (221003) -->

<%! String orgName = "TCS" ; %>

Hello <%= orgName.toString() %>

<br></br>

Today's date is <%= new java.util.Date().toString() %>

<%-- Display a message based on day of the week --%>

<br></br>

<% java.util.Calendar now = java.util.Calendar.getInstance(); %>

<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
      <p> Today is weekend</p>
<% } else { %>
      <p> Today is not weekend</p>
<% } %>

</body>
</html>
```

Following the generated servlet for above mentioned JSP. Code related to points that we discussed so far are encircled in the generated JSP. Try to relate scriptlet, expression, declaration and comments mentioned in the JSP and mentioned in the Servlet.

You can find the generated servlet in your eclipse workspace folder location under sub-folder of ".metadata" folder. For e.g. For "sm_ws" workspace location following the corresponding location of generated servlet (java class).



Following is the code of generated servlet (welcome_jsp.java).



```
welcome_jsp.java   new 2   new 3

1   package org.apache.jsp;
2
3   import javax.servlet.*;
4   import javax.servlet.http.*;
5   import javax.servlet.jsp.*;
6
7   public final class welcome_jsp extends org.apache.jasper.runtime.HttpJspBase
8       implements org.apache.jasper.runtime.JspSourceDependent {
9
10  String orgName = "TCS" ;
11   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
12
13   private static java.util.List _jspx_dependants;
14
15   private javax.el.ExpressionFactory _el_expressionfactory;
16   private org.apache.AnnotationProcessor _jsp_annotationprocessor;
17
18   public Object getDependants() {
19     return _jspx_dependants;
20   }
21
22   public void _jspInit() {
23     _el_expressionfactory = _jspxFactory.getJspApplicationContext(getServletConfig().getServletContext()).getExpressionFactory();
24     _jsp_annotationprocessor = (org.apache.AnnotationProcessor) getServletConfig().getServletContext().getAttribute(org.apache.AnnotationProcessor.class.getName());
25   }
26
27   public void _jspDestroy() {
28   }
29
30   public void _jspService(HttpServletRequest request, HttpServletResponse response)
31         throws java.io.IOException, ServletException {
32
33     PageContext pageContext = null;
34     HttpSession session = null;
35     ServletContext application = null;
36     ServletConfig config = null;
37     JspWriter out = null;
38     Object page = this;
39     JspWriter _jspx_out = null;
```

welcome_jsp.java | new 2 | new 3

```
37      JspWriter out = null;
38      Object page = this;
39      JspWriter _jspx_out = null;
40      PageContext _jspx_page_context = null;
41
42
43      try {
44        response.setContentType("text/html; charset=ISO-8859-1");
45        pageContext = _jspxFactory.getPageContext(this, request, response,
46                null, true, 8192, true);
47        _jspx_page_context = pageContext;
48        application = pageContext.getServletContext();
49        config = pageContext.getServletConfig();
50        session = pageContext.getSession();
51        out = pageContext.getOut();
52        _jspx_out = out;
53
54        out.write("\r\n");
55        out.write("<!DOCTYPE html PUBLIC \"-//W3C//DTD HTML 4.01 Transitional//EN\" \"http://www.w3.org/TR/html4/loose.dtd\">\r\n");
56        out.write("<html>\r\n");
57        out.write("<head>\r\n");
58        out.write("<meta http-equiv=\"Content-Type\" content=\"text/html; charset=ISO-8859-1\">\r\n");
59        out.write("<title>Welcome to TCS</title>\r\n");
60        out.write("</head>\r\n");
61        out.write("<body>\r\n");
62        out.write("<!-- JSP example created by Suyog Mittal (221003) -->\r\n");
63        out.write("\r\n");
64        out.write(" \r\n");
65        out.write("\r\n");
66        out.write("Hello ");
67        out.print( orgName.toString() );
68        out.write("\r\n");
69        out.write("\r\n");
70        out.write("<br></br>\r\n");
71        out.write("\r\n");
72        out.write("Today's date is ");
73        out.print( new java.util.Date().toString() );
74        out.write("\r\n");
75        out.write("\r\n");
```

welcome_jsp.java | new 2 | new 3

```
68        out.write("\r\n");
69        out.write("\r\n");
70        out.write("<br></br>\r\n");
71        out.write("\r\n");
72        out.write("Today's date is ");
73        out.print( new java.util.Date().toString() );
74        out.write("\r\n");
75        out.write("\r\n");
76        out.write("\r\n");
77        out.write("\r\n");
78        out.write("<br></br>\r\n");
79        out.write("\r\n");
80    java.util.Calendar now = java.util.Calendar.getInstance();
81        out.write("\r\n");
82        out.write("\r\n");
83    if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) {
84        out.write("\r\n");
85        out.write("      <p> Today is weekend</p>\r\n");
86    } else {
87        out.write("\r\n");
88        out.write("      <p> Today is not weekend</p>\r\n");
89    }
90        out.write("\r\n");
91        out.write("\r\n");
92        out.write("</body>\r\n");
93        out.write("</html>");
94      } catch (Throwable t) {
95        if (!(t instanceof SkipPageException)){
96          out = _jspx_out;
97          if (out != null && out.getBufferSize() != 0)
98            try { out.clearBuffer(); } catch (java.io.IOException e) {}
99          if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
100       }
101     } finally {
102       _jspxFactory.releasePageContext(_jspx_page_context);
103     }
104   }
105 }
106
```

1.1.7 Implicit Objects

JSP supports some automatically defined variables, which are also called implicit objects. There are ***9 jsp implicit objects***. These objects are created by the web container that are available to all the jsp pages.

A list of the 9 implicit objects is given below:

Objects	Type(Class)	Description
request	*HttpServletRequest*	This is the *HttpServletRequest* object associated with the request.
response	*HttpServletResponse*	This is the *HttpServletResponse* object associated with the response to the client.
out	*JspWriter*	This is the *PrintWriter* object used to send output to the client.
session	*HttpSession*	This is the *HttpSession* object associated with the request.
application	*ServletContext*	This is the *ServletContext* object associated with application context.
config	*ServletConfig*	This is the *ServletConfig* object associated with the page.
pageContext	*PageContext*	This encapsulates use of server-specific features like higher performance JspWriters.
page	*Object*	This is simply a synonym for this, and is used to call the methods defined by the translated servlet class.
Exception	*Throwable*	The Exception object allows the exception data to be accessed by designated JSP.

Details of each implicit object is mentioned below.

a) Request:

The request object is an instance of a *javax.servlet.http.HttpServletRequest* object. Each time a client requests a page the JSP engine creates a new object to represent that request.

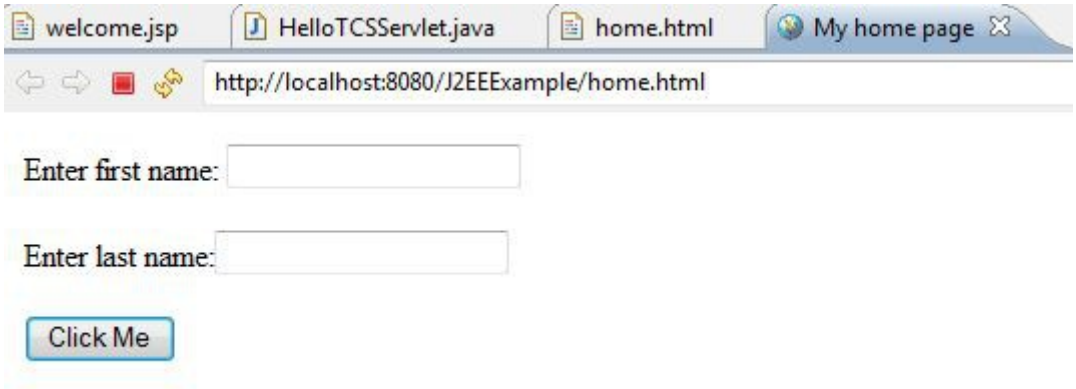
The request object provides methods to get HTTP header information including form data, cookies, HTTP methods etc.

Example-1 Below examples contain supporting files as well such as HTMLs.

Snippet of home.html which is submitting request to welcome.jsp

```
welcome.jsp    HelloTCSServlet.java    home.html
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>My home page</title>
</head>
<body>
<form name="myform" action="welcome.jsp" method="get">

Enter first name: <input type="text" name="fname" ></input>
<br></br>
Enter last name:<input type="text" name="lname" ></input>
<br></br>
<input type="submit" name="submit" value="Click Me"></input>

</form>
</body>
</html>
```

Output of home.html which is submitting request to welcome.jsp



Snippet of welcome.jsp

```
welcome.jsp   HelloTCSServlet.java   home.html   Welcome to TCS
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Welcome to TCS</title>
</head>
<body>
<!-- JSP example created by Suyog Mittal (221003) -->


Hello <%= request.getParameter("fname") %> <%= request.getParameter("lname") %>

<br></br>

Today's date is <%= new java.util.Date().toString() %>

<%-- Display a message based on day of the week --%>

<br></br>

<% java.util.Calendar now = java.util.Calendar.getInstance(); %>

<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
      <p> Today is weekend</p>
<% } else { %>
      <p> Today is not weekend</p>
<% } %>

</body>
</html>
```

output of welcome.jsp

```
welcome.jsp   HelloTCSServlet.java   home.html   Welcome to TCS
http://localhost:8080/J2EEExample/welcome.jsp?fname=Suyog&lname=Mittal&submit=Click+Me

Hello Suyog Mittal

Today's date is Wed Jun 04 17:20:26 IST 2014

Today is not weekend
```

b) response:

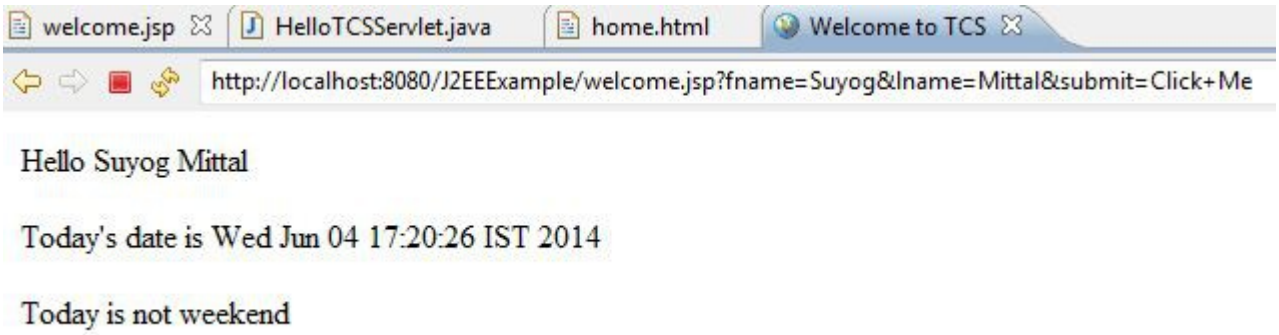
The response object is an instance of a *javax.servlet.http.HttpServletRequest* object. Just as the server creates the request object, it also creates an object to represent the response to the client.
The response object also defines the interfaces that deal with creating new HTTP headers. Through this object the JSP programmer can add new cookies or date stamps, HTTP status codes etc.

Example-1 The below example would display current system time after every 5 seconds. "response" object is used to call setIntHeader() method of to

set Refresh header to set refresh interval as 5 seconds. Its an example of auto refresh.

```
welcome.jsp   HelloTCSServlet.java   home.html   Welcome to TCS   Welcome to TCS
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Welcome to TCS</title>
</head>
<body>
<!-- JSP example created by Suyog Mittal (221003) -->

<%  response.setIntHeader("Refresh", 5); %>


Hello <%= request.getParameter("fname") %> <%= request.getParameter("lname") %>

<br></br>

Today's date is <%= new java.util.Date().toString() %>

<%-- Display a message based on day of the week --%>

<br></br>

<% java.util.Calendar now = java.util.Calendar.getInstance(); %>

<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
      <p> Today is weekend</p>
<% } else { %>
      <p> Today is not weekend</p>
<% } %>

</body>
</html>
```

Output-



c) out:

The out implicit object is an instance of a javax.servlet.jsp.JspWriter object and is used to send content in a response.

The JspWriter object contains most of the same methods as the java.io.PrintWriter class. However, JspWriter has some additional methods designed to deal with buffering. Unlike the PrintWriter object, JspWriter throws IOExceptions.

Example-1

```
<html>
<body>
<% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
</body>
</html>
```

d) session:

The session object is an instance of javax.servlet.http.HttpSession and behaves exactly the same way that session objects behave under Java Servlets.

The session object is used to track client session between client requests.

Example-1 Suporting files are mentioned in the following example such as html, servlets.

Snippet of home.html.

welcome.jsp | HelloTCSServlet.java | home.html | Internet Explorer cannot display the webpage | web.xml

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>My home page</title>
</head>
<body>
<form name="myform" action="HelloTCSServlet" method="get">

Enter first name: <input type="text" name="fname" ></input>
<br></br>
Enter last name:<input type="text" name="lname" ></input>
<br></br>
<input type="submit" name="submit" value="Click Me"></input>

</form>
</body>
</html>
```

Snippet of HelloTCSServlet.java

welcome.jsp | HelloTCSServlet.java | home.html | Apache Tomcat/6.0.32 - Error report

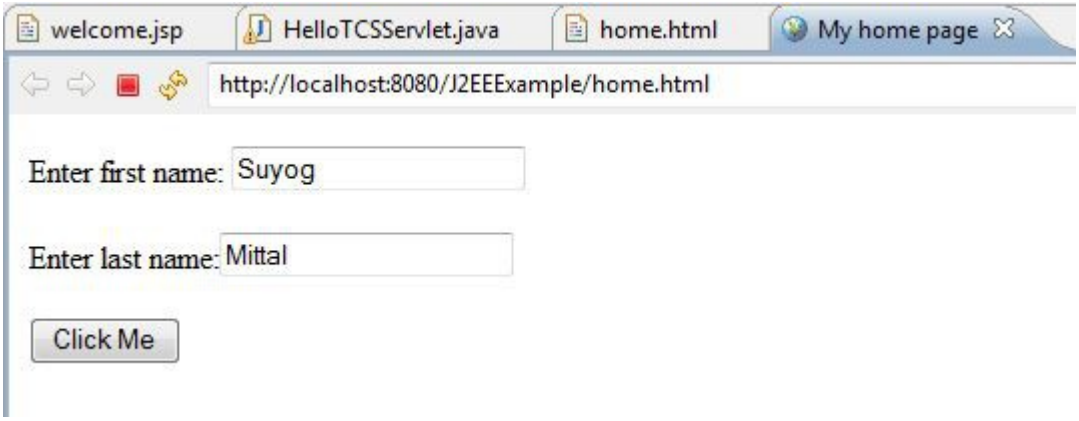
```java
21      */
22     public HelloTCSServlet() {
23         super();
24         // TODO Auto-generated constructor stub
25     }
26
27     /**
28      * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse
29      *      response)
30      */
31     protected void doGet(HttpServletRequest request,
32             HttpServletResponse response) throws ServletException, IOException {
33
34
35         HttpSession session = request.getSession(true);
36         StringBuilder name = new StringBuilder();
37         name.append(request.getParameter("fname"));
38         name.append(" ");
39         name.append(request.getParameter("lname"));
40
41         session.setAttribute("name", name);
42
43         request.getRequestDispatcher("welcome.jsp").forward(request, response);
44
45     }
46
```

Snippet of welcome.jsp

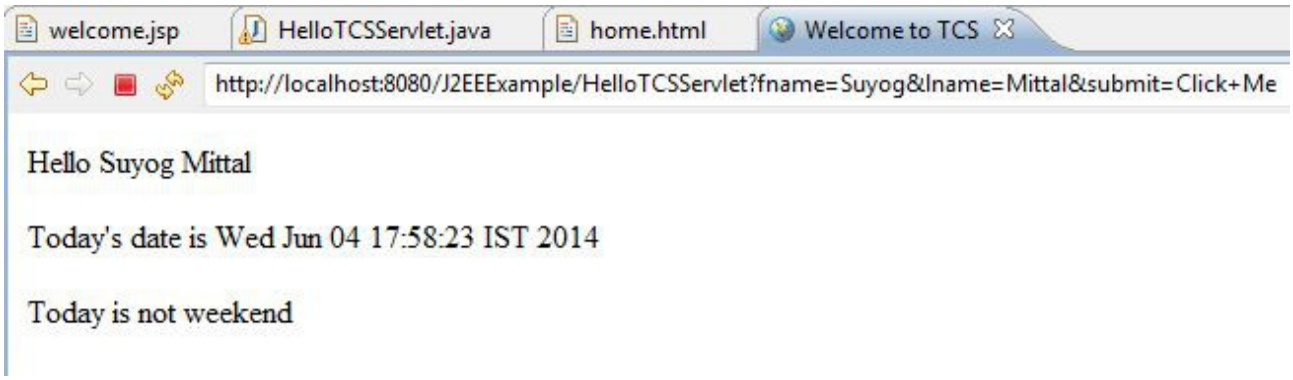
welcome.jsp | HelloTCSServlet.java | home.html

```jsp
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>Welcome to TCS</title>
8</head>
9<body>
10<!-- JSP example created by Suyog Mittal (221003) -->
11
12<%  response.setIntHeader("Refresh", 5); %>
13
14
15Hello <%= session.getAttribute("name") %>
16
17<br></br>
18
19Today's date is <%= new java.util.Date().toString() %>
20
21<%-- Display a message based on day of the week --%>
22
23<br></br>
24
25<% java.util.Calendar now = java.util.Calendar.getInstance(); %>
26
27<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
28      <p> Today is weekend</p>
29<% } else { %>
30      <p> Today is not weekend</p>
31<% } %>
32
33</body>
34</html>
```

Output of home.html



Response of Click Me



e) application:

The application object is direct wrapper around the ServletContext object for the generated Servlet and in reality an instance of a javax.servlet.ServletContext object.

This object is a representation of the JSP page through its entire lifecycle. This object is created when the JSP page is initialized and will be removed when the JSP page is removed by the jspDestroy() method.

By adding an attribute to application, you can ensure that all JSP files that make up your web application have access to it.

Example-1 Suporting files are mentioned in the following example such as html, servlets.

-home.html

welcome.jsp | HelloTCSServlet.java | web.xml | home.html

```
1<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
2<html>
3<head>
4<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
5<title>My home page</title>
6</head>
7<body>
8<form name="myform" action="HelloTCSServlet" method="get">
9
10Enter first name: <input type="text" name="fname" ></input>
11<br></br>
12Enter last name:<input type="text" name="lname" ></input>
13<br></br>
14<input type="submit" name="submit" value="Click Me "></input>
15
16</form>
17
18
19</body>
20</html>
```

Web.xml

welcome.jsp | HelloTCSServlet.java | web.xml | home.html

```
1<?xml version="1.0" encoding="UTF-8"?>
2<web-app id="WebApp_ID" version="2.4"
3    xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4    xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
5    <display-name>J2EEExample</display-name>
6
7    <servlet>
8        <description>
9        </description>
10       <display-name>HelloTCSServlet</display-name>
11       <servlet-name>HelloTCSServlet</servlet-name>
12       <servlet-class>com.tcs.ilp.HelloTCSServlet</servlet-class>
13   </servlet>
14
15
16   <servlet-mapping>
17       <servlet-name>HelloTCSServlet</servlet-name>
18       <url-pattern>/HelloTCSServlet</url-pattern>
19   </servlet-mapping>
20   <context-param>
21       <param-name>appVersion</param-name>
22       <param-value>01</param-value>
23   </context-param>
24
25   <welcome-file-list>
26       <welcome-file>index.html</welcome-file>
27       <welcome-file>index.htm</welcome-file>
28       <welcome-file>index.jsp</welcome-file>
29       <welcome-file>default.html</welcome-file>
30       <welcome-file>default.htm</welcome-file>
31       <welcome-file>default.jsp</welcome-file>
32   </welcome-file-list>
33</web-app>
```

-welcome.jsp

welcome.jsp | HelloTCSServlet.java | web.xml | home.html

```
2     pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>Welcome to TCS</title>
8</head>
9<body>
10<!-- JSP example created by Suyog Mittal (221003) -->
11
12<%  response.setIntHeader("Refresh", 5); %>
13
14
15Hello <%= session.getAttribute("name") %>
16
17<br></br>
18
19Today's date is <%= new java.util.Date().toString() %>
20
21<%-- Display a message based on day of the week --%>
22
23<br></br>
24
25Application Version is <%= application.getInitParameter("appVersion") %>
26
27
28<% java.util.Calendar now = java.util.Calendar.getInstance(); %>
29
30<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
31      <p> Today is weekend</p>
32<% } else { %>
33      <p> Today is not weekend</p>
34<% } %>
35
36</body>
37</html>
```

Response of welcome.jsp

welcome.jsp | HelloTCSServlet.java | web.xml | home.html | Welcome to TCS

http://localhost:8080/J2EEExample/HelloTCSServlet?fname=suyog&lname=mittal&submit=Click+

Hello suyog mittal

Today's date is Thu Jun 05 11:43:40 IST 2014

Application Version is 01

Today is not weekend

f) config:

The config object is an instantiation of javax.servlet.ServletConfig and is a direct wrapper around the ServletConfig object for the generated servlet. This object allows the JSP programmer access to the Servlet or JSP engine initialization parameters such as the paths or file locations etc.

Example-1 Suporting files are also mentioned in the following example such as html, servlets and web.xml.

-home.html

welcome.jsp | HelloTCSServlet.java | web.xml | home.html

```
1<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
2<html>
3<head>
4<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
5<title>My home page</title>
6</head>
7<body>
8<form name="myform" action="HelloTCSServlet" method="get">
9
10Enter first name: <input type="text" name="fname" ></input>
11<br></br>
12Enter last name:<input type="text" name="lname" ></input>
13<br></br>
14<input type="submit" name="submit" value="Click Me "></input>
15
16</form>
17
18
19</body>
20</html>
```

Web.xml

welcome.jsp | HelloTCSServlet.java | web.xml | home.html | Welcome to TCS

```
1<?xml version="1.0" encoding="UTF-8"?>
2<web-app id="WebApp_ID" version="2.4"
3    xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001/XML
4    xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/n
5    <display-name>J2EEExample</display-name>
6
7    <servlet>
8        <description>
9        </description>
10       <display-name>HelloTCSServlet</display-name>
11       <servlet-name>HelloTCSServlet</servlet-name>
12       <servlet-class>com.tcs.ilp.HelloTCSServlet</servlet-class>
13   </servlet>
14   <servlet>
15       <servlet-name>welcome</servlet-name>
16       <jsp-file>/welcome.jsp</jsp-file>
17       <init-param>
18           <param-name>jspSpec</param-name>
19           <param-value>2.2</param-value>
20       </init-param>
21   </servlet>
22   <servlet-mapping>
23       <servlet-name>welcome</servlet-name>
24       <url-pattern>/welcome.jsp</url-pattern>
25   </servlet-mapping>
26
27   <servlet-mapping>
28       <servlet-name>HelloTCSServlet</servlet-name>
29       <url-pattern>/HelloTCSServlet</url-pattern>
30   </servlet-mapping>
31   <context-param>
32       <param-name>appVersion</param-name>
33       <param-value>01</param-value>
34   </context-param>
35
```

-welcome.jsp

```
welcome.jsp   HelloTCSServlet.java   web.xml   home.html   Welcome to TCS
 3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
 4<html>
 5<head>
 6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
 7<title>Welcome to TCS</title>
 8</head>
 9<body>
10<!-- JSP example created by Suyog Mittal (221003) -->
11
12<%  response.setIntHeader("Refresh", 5); %>
13
14
15Hello <%= session.getAttribute("name") %>
16
17<br></br>
18
19Today's date is <%= new java.util.Date().toString() %>
20
21<%-- Display a message based on day of the week --%>
22
23<br></br>
24
25Application Version is  <%= application.getInitParameter("appVersion") %>
26    <br></br>
27Specification version is  <%= config.getInitParameter("jspSpec") %>
28
29<% java.util.Calendar now = java.util.Calendar.getInstance(); %>
30
31<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
32      <p> Today is weekend</p>
33<% } else { %>
34      <p> Today is not weekend</p>
35<% } %>
36
37</body>
38</html>
```

Response of welcome.jsp

g) pageContext:

In JSP, *pageContext* is an implicit object of type *PageContext* class.The *pageContext* object can be used to set,get or remove attribute from one of the following scopes:

- page (*PageContext.PAGE_SCOPE*)
- request (*PageContext.REQUEST_SCOPE*)
- session (*PageContext.SESSION_SCOPE*)
- application (*PageContext.APPLICATION_SCOPE*)

In JSP, page scope is the default scope

Example-1

home.html

welcome.jsp | HelloTCSServlet.java | web.xml | home.html | process.jsp

```
1<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.c
2<html>
3<head>
4<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
5<title>My home page</title>
6</head>
7<body>
8<form name="myform" action="HelloTCSServlet" method="get">
9
10Enter first name: <input type="text" name="fname" ></input>
11<br></br>
12Enter last name:<input type="text" name="lname" ></input>
13<br></br>
14<input type="submit" name="submit" value="Click Me "></input>
15
16</form>
17
18
19</body>
20</html>
```

HelloTCSServlet.java

welcome.jsp | HelloTCSServlet.java | web.xml | home.html | process.jsp

```java
22    public HelloTCSServlet() {
23        super();
24        // TODO Auto-generated constructor stub
25    }
26
27    /**
28     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse
29     *      response)
30     */
31    protected void doGet(HttpServletRequest request,
32            HttpServletResponse response) throws ServletException, IOException {
33
34
35        HttpSession session = request.getSession(true);
36        StringBuilder name = new StringBuilder();
37        name.append(request.getParameter("fname"));
38        name.append(" ");
39        name.append(request.getParameter("lname"));
40
41        session.setAttribute("name", name);
42
43        request.getRequestDispatcher("welcome.jsp").forward(request, response);
44
45    }
46
47    /**
48     * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse
49     *      response)
50     */
51    protected void doPost(HttpServletRequest request,
52            HttpServletResponse response) throws ServletException, IOException {
53        // TODO Auto-generated method stub
54    }
55
56 }
```

welcome.jsp

welcome.jsp | HelloTCSServlet.java | web.xml | home.html | process.jsp

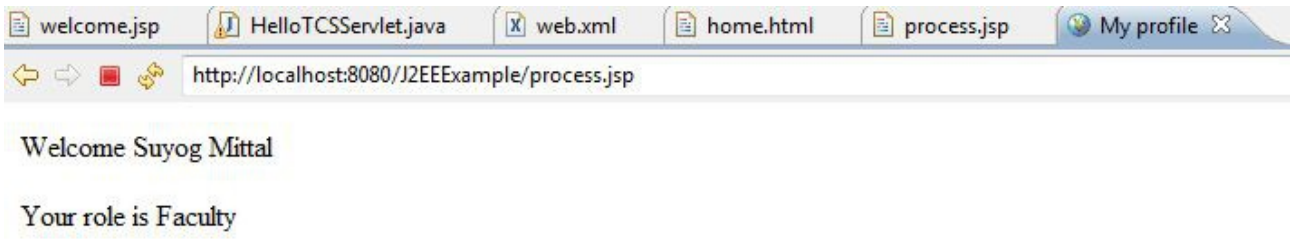
```
8</head>
9<body>
10<!-- JSP example created by Suyog Mittal (221003) -->
11
12<%  response.setIntHeader("Refresh", 5); %>
13
14
15Hello <%= session.getAttribute("name") %>
16
17<br></br>
18
19Today's date is <%= new java.util.Date().toString() %>
20
21<%-- Display a message based on day of the week --%>
22
23<br></br>
24
25Application Version is  <%= application.getInitParameter("appVersion") %>
26    <br></br>
27Specification version is   <%= config.getInitParameter("jspSpec") %>
28
29<% java.util.Calendar now = java.util.Calendar.getInstance(); %>
30
31<% if (now.get(java.util.Calendar.DAY_OF_WEEK) == 1 | now.get(java.util.Calendar.DAY_OF_WEEK) >5) { %>
32      <p> Today is weekend</p>
33<% } else { %>
34      <p> Today is not weekend</p>
35<% } %>
36
37
38<% pageContext.setAttribute("role", "Faculty",pageContext.SESSION_SCOPE ); %>
39
40<a href="process.jsp">Find role</a>
41
42</body>
43</html>
```

process.jsp

welcome.jsp | HelloTCSServlet.java | web.xml | home.html | process.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>My profile</title>
8</head>
9<body>
10Welcome <%= session.getAttribute("name") %>
11<br></br>
12Your role is <%= pageContext.getAttribute("role",pageContext.SESSION_SCOPE ) %>
13</body>
14</html>
```

Response of process.jsp



h) page:

The page implicit object is of type Object and it is assigned a reference to the servlet object that is executing the _jspService() method. Page is the instance of the JSP page's generated servlet processing the current request.

It can be used to access any method or variable defined in the generated servlet. It can be thought as direct synonym for this keyword. It is written as:

Object page=this;

For using this object it must be cast to Servlet type.For example:

<% (HttpServlet)page.log("message"); %>

Since, it is of type Object it is less used because you can use this object directly in jsp.For example:

<% this.log("message"); %>

i) exception:

The exception object is a wrapper containing the exception thrown from the previous page. It is typically used to generate an appropriate response to the error condition.

Example-1

Detail example is shown in section 1.1.12 JSP Exception Handling.

```
<%@ page isErrorPage="true" %>
<html>
<body>

Sorry following exception occured: <%= exception %>

</body>
</html>
```

1.1.8 JSP Directives

The jsp directives are messages that tells the web container how to translate a JSP page into the corresponding servlet.There are three types of directives:

- page directive
- include directive
- taglib directive

The page Directive:

The page directive is used to provide instructions to the container that pertain to the current JSP page. You may code page directives anywhere in your JSP page. By convention, page directives are coded at the top of the JSP page. Following is the basic syntax of page directive:

```
<%@ page attribute="value" %>
```

Following is the list of attributes associated with page directive:

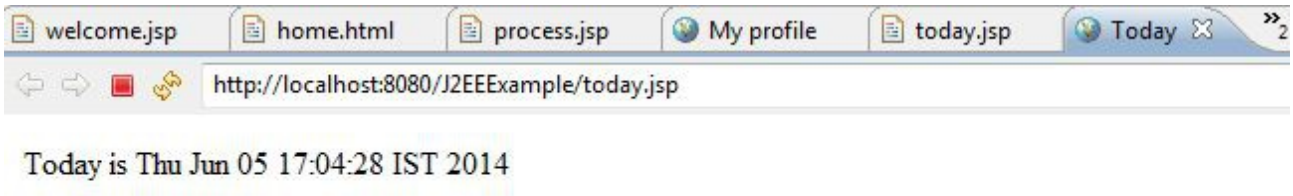
Attribute	Purpose
buffer	Specifies a buffering model for the output stream.
autoFlush	Controls the behavior of the servlet output buffer.
contentType	Defines the character encoding scheme.
errorPage	Defines the URL of another JSP that reports on Java unchecked runtime exceptions.
isErrorPage	Indicates if this JSP page is a URL specified by another JSP page's errorPage attribute.

extends	Specifies a superclass that the generated servlet must extend
import	Specifies a list of packages or classes for use in the JSP as the Java import statement does for Java classes.
info	Defines a string that can be accessed with the servlet's getServletInfo() method.
isThreadSafe	Defines the threading model for the generated servlet.
language	Defines the programming language used in the JSP page.
session	Specifies whether or not the JSP page participates in HTTP sessions
isELIgnored	Specifies whether or not EL expression within the JSP page will be ignored.
isScriptingEnabled	Determines if scripting elements are allowed for use.

Example-1: import
today.jsp

```
1<%@ page import="java.util.Date" language="java" contentType="text/html; charset=ISO-8859-1"
2
3    pageEncoding="ISO-8859-1"%>
4
5<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
6<html>
7<head>
8<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
9<title>Today</title>
10</head>
11<body>
12
13Today is <%= new Date() %>
14
15</body>
16</html>
```

Response



Few of these attributes are explained in next section like isErrorPage and errorPage are explianed in jsp exception handling section.

1.1.9 The Include Directive

You use the include directive to include the content of another file in the current JSP page. You can use multiple include directives in a JSP page.You may code include directives anywhere in your JSP page. Modularizing a particular content into an include file is useful if that content is used by different pages or used by a page in different places.In servlet if we want to modularize a specific content then we create a relevant method and write our logic in that method.

The syntax of the include directive is as follows:

```
<%@ include file="url"%>
```

where the space between @ and include is optional and url represents the relative path to an include file. If url begins with a forward slash (/), it is interpreted as an absolute path on the server. If it does not, it is interpreted as relative to the current JSP page.

The JSP translator translates the include directive by replacing the directive with the content of the include file. In other words, if you have written the footer.jsp file which is present below.

footer.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>Insert title here</title>
8</head>
9<body>
10
11<hr/>
12&copy;2014 TCS
13<hr/>
14</body>
15</html>
```

And, you have the main.jsp page

main.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://
4<html>
5<head><title>Including a file</title></head>
6<body>
7This is the included content: <hr/>
8<%@ include file="footer.jsp"%>
9</body>
10</html>
```

Using the include directive in the main.jsp page has the same effect as writing the following JSP page.

<html>

<head><title>Including a file</title></head>

<body>

This is the included content: <hr/>

<hr/>

©2014 TCS

<hr/>

</body>

</html>

For the above include directive to work, the footer.jsp file must reside in the same directory as the including page.

1.1.10 Include Action or <jsp:include>

The include action is used to include another resource dynamically. You can include another JSP page, a servlet, or a static HTML page. For example, the jspIncludeExample.jsp page in mentioned below uses the include action(<jsp:include>) to include the menu.jsp page(menu.jsp page is another jsp page).

As you have already seen the RequestDispatcher.include method in servlet and also you know that each jsp is converted to JSP so here when you write jsp:include tag it is actually include method of requestdispatcher in servlet.

jspIncludeExample.jsp | menu.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/
4<html>
5<head>
6<title>Include action or jsp:include</title>
7</head>
8<body>
9<jsp:include page="menu.jsp">
10    <jsp:param name="text" value="How are you?"/>
11</jsp:include>
12</body>
13</html>
```

It is important that you understand the difference between the include directive which we have just seen (1.2.1) and the include action or jsp:include.

With the include directive(1.2.1), inclusion occurs at page translation time, i.e. when the JSP container translates the page into a generated servlet.

It is important that you understand the difference between the include directive which we have just seen (1.2.1) and the include action or jsp:include. With the include directive(1.2.1), inclusion occurs at page translation time, i.e. when the JSP container translates the page into a generated servlet.

- With the include action, inclusion occurs at request time. As such, you can also pass parameters using the include action, but not the include directive. To pass the parameters from index to file page we are using <jsp:param> action tag.

  Example of the same is given below

In the registration.jsp page we are including includeMe.jsp page.

Code of registration.jsp is

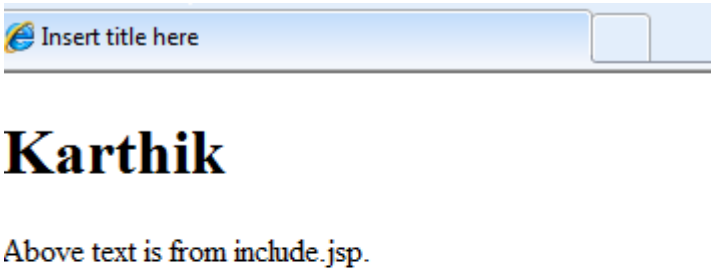
```
<%@ page language="java" contentType="text/htm
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 T
<html>
<head>
<meta http-equiv="Content-Type" content="text/
<title>Insert title here</title>
</head>
<body>
<jsp:include page="includeMe.jsp">
  <jsp:param name="UserName" value="Karthik"/>
</jsp:include>
    Above text is from include.jsp.
</body>
</html>
```

Here in registration.jsp you can see that using <jsp:include> tag "includeMe.jsp" page is include and then using <jsp:param> tag we are passing "UserName" parameter to includeMe.jsp page.

Code of includeMe.jsp is

```
<%@ page language="java" contentType="text/html; char
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transiti
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; c
<title>Include Me</title>
</head>
<body>
<h1><%=request.getParameter("UserName") %> </h1>
</body>
</html>
```

Now when you run registration.jsp page you will see the below mentioned output.



- The second difference is that with the include directive(1.2.1), the file extension of the included resource does not matter. With the include action or jsp:include, the file extension must be jsp for it to be processed as a JSP page.

1.1.11 Forward Action or <jsp:forward>:

The forward action or jsp:forward forwards the current page to a different

resource. For example, the following forward action or jsp:forward forwards the current page to the login.jsp page.

As you have already seen the RequestDispatcher.forward method in servlet and also you know that each jsp is converted to JSP so here when you write jsp:forward tag it is actually forward method of requestdispatcher in servlet.

jspIncludeExample.jsp | menu.jsp | *jspForwardExample.jsp

```
1<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2    pageEncoding="ISO-8859-1"%>
3<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/
4<html>
5<head>
6<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7<title>jsp forward Example</title>
8</head>
9<body>
10<jsp:forward page="jspf/login.jsp">
11    <jsp:param name="text" value="Please login"/>
12</jsp:forward>
13</body>
14</html>
```

1.1.12 JSP Exception Handling:

Exception handling is well supported in JSP. Java code can be handled using the try statement, however you can also specify a page that will be displayed when any of the pages in the application encounters an uncaught exception. In such events, users will see a well designed page that explains what happened, and not an error message that makes them frown.

You make a JSP page an error page by using the isErrorPage attribute of the page directive. The value of the attribute must be true. Below mentioned example shows such an error handler page which is myApplicationErrorPage.jsp(name of the jsp page).This is the page which will be displayed when any exception will occur in any of the jsp page in application.

myApplicationErrorPage.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http:
<%@page isErrorPage="true"%>
<html>
<head><title>Error</title></head>
<body>
An error has occurred. <br/>
Error message:
<%
    out.println(exception.toString());
%>
</body>
</html>
```

Other pages that need protection against uncaught exceptions will have to use the errorPage attribute of the page directive, mentioning the path to the error handling page as the value. For example, the pageWithSomeError.jsp page mentioned below uses myApplicationErrorPage.jsp

pageWithSomeError.jsp is mentioned below and this is the page which is throwing an exception..

myApplicationErrorPage.jsp | pageWithSomeError.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://ww

<%@page errorPage="myApplicationErrorPage.jsp"%>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>This page throws error</title>
</head>
<body>
Deliberately throw an exception
<%
    Integer.parseInt("Throw me");
%>
</body>
</html>
```

If you run the pageWithSomeError.jsp page, it will throw an exception.

If you have not included this line <%@page errorPage="myApplicationErrorPage.jsp"%> then you will get below mentioned

page

Apache Tomcat/6.0.32 - Error report

HTTP Status 500 -

---

type Exception report

message

description The server encountered an internal error () that prevented it from fulfilling this request.

exception

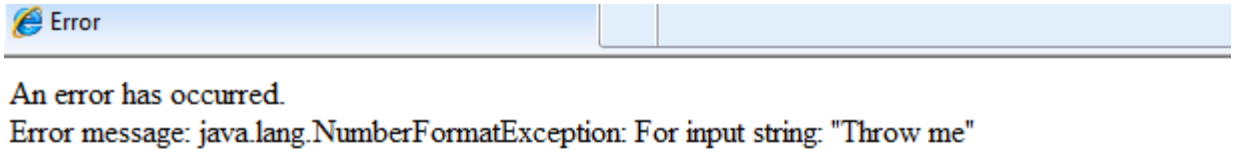
```
org.apache.jasper.JasperException: An exception occurred processing JSP page /pageWithSomeError.jsp at line 14

11: <body>
12: Deliberately throw an exception
13: <%
14:     Integer.parseInt("Throw me");
15: %>
16: </body>
17: </html>


Stacktrace:
        org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:519)
        org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:428)
        org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:313)
        org.apache.jasper.servlet.JspServlet.service(JspServlet.java:260)
        javax.servlet.http.HttpServlet.service(HttpServlet.java:717)
```

root cause

```
java.lang.NumberFormatException: For input string: "Throw me"
        java.lang.NumberFormatException.forInputString(NumberFormatException.java:48)
        java.lang.Integer.parseInt(Integer.java:447)
        java.lang.Integer.parseInt(Integer.java:497)
        org.apache.jsp.pageWithSomeError_jsp._jspService(pageWithSomeError_jsp.java:65)
        org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
        javax.servlet.http.HttpServlet.service(HttpServlet.java:717)
        org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:386)
        org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:313)
        org.apache.jasper.servlet.JspServlet.service(JspServlet.java:260)
        javax.servlet.http.HttpServlet.service(HttpServlet.java:717)
```

note The full stack trace of the root cause is available in the Apache Tomcat/6.0.32 logs.

However, you will not see above mentioned page. Instead, the content of the myApplicationErrorPage.jsp page is displayed which is mentioned below.



1.1.13 UseBean Action or UseBean:

The useBean action is used to instantiate a new JavaBean for use later on the JSP page. The getProperty and setProperty actions are used to get and set properties that have been defined within a JavaBean.

It is javabean convention that whichever attributes you define in javabean must have getters and setters method.

The JavaBean actions fulfill a crucial role in the component model of JSP pages. By placing all business logic within JavaBeans, and placing all presentation

logic into the JSP page, Web applications can be written as separate, reusable, and scaleable components. The JavaBean actions are the key to tying together the JavaBean business logic and JSP page presentation logic.

This action creates a scripting variable associated with a Java object. It was one of the earliest efforts to separate presentation and business logic. Thanks to other technologies such as custom tags and the Expression Language, useBean is now rarely used.

myApplicationErrorPage.jsp | pageWithSomeError.jsp | myUseBeanPage.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://w
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>jsp usebean example</title>
</head>
<body>
<jsp:useBean id="dateOfToday" class="java.util.Date"/>
<%=dateOfToday%>
</body>
</html>
```

The action will be translated into this servlet code at server side.

```
java.util.Date dateOfToday = null;
dateOfToday = (java.util.Date) _jspx_page_context.getAttribute("dateOfToday",
        javax.servlet.jsp.PageContext.REQUEST_SCOPE);
if (dateOfToday== null) {
    dateOfToday = new java.util.Date();
    _jspx_page_context.setAttribute("dateOfToday", dateOfToday,
            javax.servlet.jsp.PageContext.REQUEST_SCOPE);
}
```

Running this page prints the current date and time in your browser

a) setProperty and getProperty

The setProperty action sets a property in a Java object and getProperty prints a Java object's property. As an example, the getSetPropertyUseBeanExample.jsp page below sets and gets the firstName property of an instance of the Customer class, defined below

pageWithSomeError.js | myUseBeanPage.jsp | getSetPropertyUseBea | Custom

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http
<html>
<head>
<title>getProperty and setProperty jspusebean example</title>
</head>
<body>
<jsp:useBean id="customer" class="com.tcs.lms.bean.Customer"/>
<jsp:setProperty name="customer" property="firstName"
      value="pankaj"/>
First Name: <jsp:getProperty name="customer" property="firstName"/>
</body>
</html>
```

```java
package com.tcs.lms.bean;

public class Customer {
    private String customerId;
    private String firstName;
    private String lastName;
    /**
     * @return the customerId
     */
    public String getCustomerId() {
        return customerId;
    }
    /**
     * @param customerId the customerId to set
     */
    public void setCustomerId(String customerId) {
        this.customerId = customerId;
    }
    /**
     * @return the firstName
     */
    public String getFirstName() {
        return firstName;
    }
    /**
     * @param firstName the firstName to set
     */
    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }
    /**
     * @return the lastName
     */
    public String getLastName() {
        return lastName;
    }
    /**
     * @param lastName the lastName to set
     */
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }


}
```

1.1.14 JSP Single Thread Model

As you have seen in case of servlet when multiple threads are trying to access a single servlet instance it may drag you to abnormal state of servlet class and to avoid that servlet implements singleThreadModel interface.

If same situation occurs in jsp then this is achieved by simply turning off threading for the entire page. Turning off threading is a poor option and should be avoided in most situations, because it avoids the potential problems by sacrificing many advantages.The simplest method for gaining thread safety is also the least efficient.

JSP provides a means to turn off threading through the page directive attribute(remember we have gone through page directive ??) isThreadSafe='false'. This forces the page to be created under the SingleThreadModel, which allows only one request to be handled by the page at any time.

```
<%@ page language="java" contentType
    pageEncoding="ISO-8859-1"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD H
<%@page isThreadSafe="false"%>
<html>
<head>
<meta http-equiv="Content-Type" cont
<title>Insert title here</title>
</head>
<body>

</body>
</html>
```

However,it is not recommended to use this because in this case for each request for a jsp, new jsp instance will be created which is a memory overhead. Also JSP is used as view in terms of MVC pattern,so if you use JSP to write business logic then that is not right approach!

1.1.15 Summary

- JSP is the second technology for building web applications in Java, invented
- to complement Servlet technology and not to replace it.
- JSP has threee syntactic elements to support dynamic content generation

    - Scriptlet tag (<% .... %>)
    - Expression tag (<%= ..... %>)
    - Declaration tag (<%! .... %>)
- Implicits objects are available to developers to perform various operations.
- The jsp directives are messages that tells the web container how to translate a JSP page into the corresponding servlet.There are three types of directives:
    - page directive
    - include directive
    - taglib directive
- JSP provides inbuilt mechanism to handle exceptions.
- By now you've learned how JSP works and how to write JSP pages.